CSE 207 Data Structures and Algorithms II

3 hours in a week, 3.00 credits

Graph algorithms: MST algorithms, shortest path algorithms, maximum flow and maximum bipartite matching;
Lower bound theory;
Advanced data Structures: balanced binary search trees (AVL trees, red-black trees, splay trees, ~~skip lists~~), advanced heaps (~~Fibonacci heaps~~, binomial heaps);
Amortized analysis;
Hashing;
NP-completeness;
NP-hard and NP-complete problems;
Coping with hardness: backtracking, branch and bound, approximation algorithms;
~~String matching algorithms~~;
~~FFT and its applications~~.

Related OCW courses:
6.046J | Spring 2015
6.006 | Spring 2020
6.006 | Fall 2011

Graph Algorithm

Breadth first search (BFS)

```
BFS(G, root):
    For each vertex v of G:
        v.parent = null
        v.dist = inf
        v.visited = false
    Initialize queue Q
    Q.enqueue(root)
    root.dist = 0
    root.visited = true
    While Q is not empty:
        u = Q.dequeue()
        For all neighbor v of u:
            If v.visited = false:
                v.parent = u
                v.dist = u.dist + 1
                v.visited = true
                Q.enqueue(v)
```

Runtime:
O(V+E)

Depth first search (DFS)

```
DFS(G):
    For each vertex v of G:
        v.parent = null
        v.startTime = inf
        v.endTime = inf
        v.visited = false
    time = 0
    For each vertex v in G:
        If v.visited = false:
            DFS_visit(v)
DFS_visit(u):
    u.visited = true
```

```
    u.startTime = ++time
    For each children v of u:
        If v.visited = false:
            v.parent = u
            DFS_visit(v)
    u.finishTime = ++time
```

Runtime:
O(V+E)

Minimum Spanning Tree (MST)

Prim's Algorithm

Simulation

```
prim(G):
    initialize priority queue Q (sort based on key)
    For each vertex v of G, set →
        v.parent = null
        v.key = inf
        Q.enqueue(v)
    choose arbitrary start vertex s
    Q.decrese_key(s) = 0
    Set ans = 0
    while Q is not empty:
        u = Q.extract_min()
        ans += u.key
        For each neighbor v of u:
            if v in Q and w(u, v) < v.key:
                Q.decrease_key(v) = w(u, v)
                v.parent = u
    return ans
```

Runtime:
$O(V).T_{extract\_min} + O(E).T_{decrease\_key}$
Array → $O(V^2)$
$T_{extract\_min} = O(V)$
$T_{decrease\_key} = O(1)$
Binary Heap → O(E logV)

$T_{extract\_min} = O(\log V)$

$T_{decrease\_key} = O(\log V)$

Fibonacci Heap → $O(E + V \log V)$

$T_{extract\_min} = (\log V)$ [amortized]

$T_{decrease\_key} = O(1)$ [amortized]

Kruskal's Algorithm

Simulation

```
krushkal(G):
    Initialize a Union-Find structure T
    For each vertex v of G:
        make_set(v)
    sort(E) in ascending order of weights
    Set ans = 0
    for e = (u, v) in E:
        if find_set(u) != find_set(v):
            T.add(e)
            ans += w(u, v)
            union(u, v)
    return ans
```

Runtime:

$T_{sort}(E) + O(V).T_{make\text{-}set} + O(E).(T_{find} + T_{union}) = O(E \log E + E.a(V)) = O(E \log E)$

$T_{make\text{-}set} = O(1)$

$T_{find} + T_{union} = O(a(V))$ [amortized]

*if weights are in range $[0, E^{O(1)}]$ → $T_{sort} = O(E)$ [counting sort]

Reverse-Delete Algorithm

```
reverseDelete(G):
    ans = 0
    sort(E) in descending order according to weights
    for e = (u, v) in E:
        remove e G
        if G is not connected:
            add e to G
            ans += w(u, v)
    return ans
```

Runtime:
$O(E \log V (\log(\log V))^3)$

Shortest Path Algorithm

Single Source Shortest Path (SSSP)

Relaxation:

```
relax(u, v, w):
    if v.distance > u.distance + w:
        v.distance = u.distance + w
        v.parent = u
```

Dijkstra's Algorithm

Simulation

```
dijkstra(G, s):
    initialize priority queue Q (sort based on distance)
    initialize set S
    for each vertex v of G, set →
        v.key = inf
        v.parent = null
        Q.enqueue(v)
    s.distance = 0
    while Q is not empty:
        u = Q.dequeue()
        S.add(u)
        for each neighbor v of u:
            relax(u, v, w(u, v))
```

Runtime:
$O(V).T_{extract\text{-}min} + O(E).T_{decrease\text{-}key}$
Array → $O(V^2)$
$T_{extract\text{-}min} = O(V)$
$T_{decrease\text{-}key} = O(1)$
Binary Heap → $O(E \log V)$
$T_{extract\text{-}min} = (\log V)$
$T_{decrease\text{-}key} = O(\log V)]$
Fibonacci Heap → $O(E + V \log V)$

$T_{extract\text{-}min}$ = (logV) [amortized]
$T_{decrease\text{-}key}$ = O(1) [amortized]

Bellman-Ford

Simulation

```
bellmanFord(G, s):
    For each vertex v of G, set →
        s.distance = 0
        v.distance = inf
        v.parent = null
    for i = 1 to |V|-1:
        for each edge (u, v) in E of G:
            relax (u, v, w)
    for each edge (u, v) in E of G:
        if v.distance > u.distance + w(u, v):
            report → there is a negative cycle
```

Runtime:
O(VE)

All Pair Shortest Path (APSP)

Dynamic programming

```
dp(G):
    for i = 1 to |V|:
        for j = 1 to |V|:
            d[i][j] = w(i, j)
    for m = 1 to |V| and m = m*2:
        for u in V:
            for v in V:
                for x in V:
                    if d[u][v] > d[u][x] + d[x][v]:
                        d[u][v] = d[u][x] + d[x][v]
```

Runtime:
$O(V^3 \log V)$

Floyd-Warshal

Simulation

```
floydWarshall(G):
    for i = 1 to |V|:
        for j = 1 to |V|:
            c[i][j] = w(i, j)
    for k = 1 to |V|:
        for u in V:
            for v in V:
                if c[i][j] > c[i][k] + c[k][v]:
                    c[i][j] = c[i][k] + c[k][v]
```

Runtime:

$O(V^3)$

Matrix multiplication

```
matrixMultiplication(G):
    for i = 1 to |V|:
        for j = 1 to |V|:
            d[i][j] = w(i, j)
    for m = 1 to |V| and m = m*2:
        for u in V:
            for v in V:
                for x in V:
                    if d[u][v] > d[u][x] + w(x, v):
                        d[u][v] = d[u][x] + w(x, v)
```

Runtime:

$O(V^3 \log V)$

Johnson's Algorithm

1. Add a new vertex s
2. Add edges from s to every vertex v in V\{s} of G, where w(s, v) = 0
3. Run Bellman-Ford once from s, in O(VE)
4. Define h(v) = δ(s, v) for all v, in V
5. Reweight every edge $w_h(u, v) = w(u, v) + h(u) - h(v)$, in O(E)
6. Run Dijkstra from every vertex v in V, in $O(VE + V^2 \log V)$
7. Reweight every pair shortest path, in $O(V^2)$

Runtime:

$O(VE + V^2 \log V)$

Maximum Flow

Ford-Fulkerson Algorithm

fordFulkerson(G, s, t):
    Initialize maxFlow = 0
    Initialize flow of each edge f(u, v) = 0
    For all edges in G (initialize residual graph G_f) set
        $c_f(u, v) = c(u, v) - f(u, v)$
    While there exists an augmenting path p from s to t in G_f:
        find the path p
        determine the amount of flow tempMin that can be sent along p
        maxFlow += tempMin
        update G_f
    return maxFlow

Runtime:
O(maxFlow*E)
Simulation

Edmonds-Karp Algorithm

edmondsKarp(G, s, t):
    Initialize maxFlow = 0
    Initialize flow of each edge f(u, v) = 0
    For all edges in G (initialize residual graph G_f) set
        $c_f(u, v) = c(u, v) - f(u, v)$
    While there exists an augmenting path, p from s to t in residual graph G_f:
        Find the path p using BFS
        Determine the amount of flow tempMin that can be sent along p
        maxFlow += tempMin
        update G_f
    return maxFlow

Runtime:
$O(VE^2)$

Maximum Bipartite Matching

```
bipartiteMatching(A, B):
    Build a flow network, G
        add a source s
        add edges from s to every vertex a in A, where c(s, a) = 1
        add edges between A and B accordingly, usually c(a, b) = 1
        add a tank t
        add edges from every vertex b in B to t, where c(b, t) = 1
    return the maximum flow of G
```

Max-Flow Min-Cut Theorem

maximum flow, f = c(S, T) for some cut S, T

```
minCut(G, s, t):
    Initialize sets S and T
    edmondsKarp(G, s, t)
    for all v in V set →
        v.visited = false
    dfs(G_f, s)
    for u in V:
        for v in V:
            if u.visited = true and v.visited = false:
                S.add(u)
                T.add(v)
    return S and T
```

Theorem:

The following are equivalent →

1. |f| = c(S, T) for some cut (S, T)
2. f is a maximum flow
3. f admits no augmenting paths

Proof:

Since $| f | \leq c(S, T)$ for any cut (S, T), the assumption that |f| = c(S, T) indicates that f is a maximum flow.

If there were an augmenting path, the flow value could be increased, contradicting the maximality of f.

Lower Bound Theory

- A lower bound for a problem is the worst-case running time of the best possible algorithm for that problem.

Amortized Analysis

Determine worst-case running time of a sequence of n data structure operations.

Aggregate Method

Just add up the cost of all the operations and then divide by the number of operations.

Accounting Method

This method allows operations →

- to store credit for future use, if its assigned amortized cost > its actual cost
- to pay for its extra actual cost using existing credit, if its assigned amortized cost < its actual cost

Data structure after i^{th} operation, D_i
Actual cost of ith operation, c_i
Amortized cost (amount we charge) of i^{th} operation, $\hat{c}_i$
When $\hat{c}_i > c_i$ we store credits in data structure D_i to pay for future operations. When $\hat{c}_i < c_i$, we consume credits in data structure D_i. Initial data structure D_0 starts with 0 credits.
The total number of credits in the data structure ≥ 0.
$\sum \hat{c}_i \geq \sum c_i$
Explanation

Potential Method

This method defines a potential function Φ that maps a data structure configuration to a value. This function Φ is equivalent to the total unused credits stored up by all past operations (the bank account balance)
Potential function: $\Phi(D_i)$ maps each data structure D_i to a real number, where $\Phi(D_0) = 0$ and $\Phi(Di) \geq 0$ for each data structure D_i
Actual cost of i^{th} operation, c_i
Amortized cost of i^{th} operation, $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$
Starting from the initial data structure D_0, the total actual cost of any sequence of n operations is at most the sum of the amortized costs.

$\sum \hat{c}_i = \sum (c_i + \Phi(D_i) - \Phi(D_{i-1})) = \sum c_i + \Phi(D_n) - \Phi(D_0) \geq \sum c_i$

Explanation

Balanced Binary Search Tree

AVL Tree

- Properly balanced tree.

```
class Node:
    int value
    int height
    Node *right
    Node *left
class BST:
    Node *root
    rightRotate(node):
        l = node.left
        lr = l.right
        l.right = node
        node.left = lr
        recalculate height of node and l
        return l
    leftRotate(node):
        r = node.right
        rl = r.left
        r.left = node
        node.right = rl
        recalculate height of node and r
        return r
    balaceFactor(node):
        return node.left.height - node.right.height
    balance(node):
        if balanceFactor(node)  > 1:
            if balanceFactor(node.left) < 0 do left-right rotation:
                node.left = leftRotation(node.left)
                node = rightRotation(node)
            else do right rotation:
```

```
                node = right rotation(node)
        else if balanceFactor(node) < -1:
                if balanceFactor(node.right) > 0 do right-left rotation:
                        node.right = rightRotation(node.right)
                        node = leftRotation(node)
                else do left rotation:
                        node = leftRotate(node)
```

Simulation 1
Simulation 2

Red-Black Tree

- Roughly balanced tree.
- Every node is either black or red.
- Null nodes are considered black.
- The root is black.
- No red node has a red child (Color Invariant).
- Number of black nodes on every path from root to leaves is the same (Height Invariant).
- The height and color invariants together imply that the longest path from the root to a leaf contains at most twice as many nodes as in the shortest path.

Height

A red-black tree with n nodes has height at most 2 log(n+1)
We first prove that the subtree rooted at any node x, with a black height of bh(x), contains at least $2^{bh(x)} – 1$ nodes
Base case: If x is a leaf and is black, then bh(x) = 1. There is at least $2^1 - 1 = 1$ nodes
If x is a leaf and is red, then bh(x) = 0. There are at least $2^0 - 1 = 0$ nodes.
Induction: For any node x with bh(x) ≥ 1, each child has a black-height of bh(x) or bh(x) -1, depending on whether x is red or black, respectively. By induction hypothesis, each child has at least $2^{bh(x)-1} – 1$ nodes.
Thus the subtree rooted at x contains $2 (2^{bh(x)-1} – 1) + 1 = 2^{bh(x)} – 1$ node.
Next, the tree has at least $2^{bh(root)} – 1$ nodes
Hence, $n \geq 2^{bh(root)} – 1$
$\Rightarrow n \geq 2^{h/2} – 1$
$\Rightarrow h \leq 2 \log (n+1)$
$\Rightarrow h = O(\log n)$

Simulation

Insertion

- Color the new node red.
- If it is root, make it black.
- If its parent is black, no problem.
- If its parent is red, problem. Fix it:

  Case 1: Uncle node is also red:
  - Make grand-parent red, parent and uncle black. If doing this makes our root node red, just make it black, this will increase the overall black height by 1.

  Case 2: Uncle is black (zig-zig) (right-right or left-left):
  If the inserted node is a leaf node, the uncle is empty (black).
  - Rotate grandparent.
  - Swap colors of parent and grandparent.

  Case 3: Uncle is black (zig-zag) (left-right or right-left):
  - Rotate parent. It will create case 2.

Simulation

Complexity:

Insertion, deletion, search → O(log n)

Splay Tree

- Roughly balanced tree
- Recently accessed elements are quick to access again by splaying these to the root
- Any m consecutive operations starting from an empty tree take at most O(m log n) time

Simulation

Insertion

Insert(T, x):

Standard insertion of x, then splay(T, x)

Alternate: Split(T, x) → Two subtrees created L and R → make a tree with x as root and L as left subtree, R as right subtree.

Simulation

Deletion

Delete(T, x):

Bottom-up approach: Standard deletion of x, then splay(T, parent(x))
Top-down approach: Splay(T, x) → then delete root → two subtrees created L and R → Join(L, R) → Splay on the maximum element of L and attach R to the new root of L. (If R is null, still need to splay on the maximum element of L and then no join. But if L is null, no need to splay or join)
Bottom-up simulation
Top-down simulation

Searching

Search(T, x):
Standard searching of x, then splay(T, x)

Splaying

Splay(T, x):

- No rotation: when x is the root node.
- Zig rotation: when x is a child of the root.
- Zig-zig rotation: left-left or right-right
- Zig-zag rotation: left-right or right-left

Perform rotation until x is the root.
Complexity:
Splaying → O(log n) [amortized]
Insertion, deletion, search → O(log n) [amortized], O(n) [worst case]

Advanced Heap

Binary Heap

Make-Heap → O(1)
isEmpty → O(1)
Insert → O(log n)
Extract-Min → O(log n) [O(1) amortized]
Decrease-Key → O(log n)
Delete → O(log n) [O(1) amortized]
Meld → O(n)
Find-Min → O(1)

D-ary Heap

Make-Heap → O(1)
isEmpty → O(1)
Insert → $O(\log_d n)$
Extract-Min → $O(d \log_d n)$
Decrease-Key → $O(\log_d n)$
Delete → $O(d \log_d n)$
Meld → O(n)
Find-Min → O(1)

Binomial Heap

Binomial tree →

Definition:

B_0: single node

B_k: A B_{k-1} linked to another B_{k-1}

B_k →

Height: k

Number of nodes: 2^k

Number of nodes at depth i: $\binom{k}{i}$

Degree: k

Deleting root yields k binomial trees → $B_0, B_1, \ldots, B_{k-1}$

Binomial Heap →

- Sequence of heap-ordered binomial trees
- At Most one tree of each degree, like binary representation has a list of roots of all the binomial trees

Meld(H, H`):

- Analogous to binary addition.

Runtime: O(log n) [O(1) amortized]

Extract-min(H):

- Find least value in the root-list
- Delete the least value, that yields broken binomial tree H`
- Meld(H, H`)

Runtime: O(log n)

Decrease-key(H, x, k):

- Set x → k
- Repeatedly exchange x with its parent until getting to the root

Runtime: O(log n)

Delete(H, x):

- Decrease-key(H, x, -inf)
- Extract-min(H)

Runtime: O(log n)

Insert(H, x):

- H` → make-heap()
- H` → insert(x)
- Meld(H, H`)

Runtime: O(log n) [O(1) amortized]

Find-min(H):

Runtime: O(log n) [O(1) amortized]

Make-Heap():

Runtime: O(1)

isEmpty(H):

Runtime: O(1)

Hashing

- Load factor, λ = N/TableSize
- Hash-table size prime
- Usually load factor < 0.5
- **Collision chance order:** linear probing ≥ quadratic probing ≥ {random probing, double hashing}
- Good for searching
- Not good for ordering

Search, Insertion → O(1) [average]

Collision resolution strategies

Chaining

- Store colliding keys in a linked list at the same hash table index

```
class HashTable:
        array of vector arr
        int size, count
        int myHash(hashObj x):
                return hash(x)%size
```

```
insert(hashObj x):
        if find(x):
                return
        arr[myHash(x)].push_back(x)
        count++
        if (count/size) > loadFactor:
                rehash()
delete(hashObj x):
        arr[myHash(x)].erase(x)
find(hashObj x):
        for el in arr[myHash(x)]:
                if el = x:
                        return true
        return false
rehash():
        copy all the elements in an array of vectors temp
        redefine arr with 2 × size
        for each hashObj x in temp:
                insert(x)
```

Unsuccessful search time: $O(\lambda)$
Successful search time: $O(\lambda/2)$

Open Addressing

- Store colliding keys elsewhere in the table

Probe sequence: sequence of slots in hashtable while searching for an element

Linear Probing

- Primary clustering

Hash Function, $h_i(x) = (h(x) + f(i)) \% \text{tableSize}$, where f(i) is a linear function of i
Expected number of probes:

Successful search = $\frac{1}{2} (1 + 1/(1 - \lambda)^2)$
Insertion or Unsuccessful search = $\frac{1}{2} (1 + 1/(1 - \lambda))$

Random probing

- Does not suffer from clustering

Expected number of probes:

Insertion or Unsuccessful search = $(1/\lambda) \ln(1/(1 - \lambda))$

Quadratic Probing

- Avoids primary clustering

- Difficult to analyze
- May cause secondary clustering
- Lazy deletion

Hash Function, $h_i(x) = (h(x) + f(i)) \% \text{tableSize}$, where f(i) is a quadratic function of i

Double Hashing

- Keep two hash functions
- Close to random hashing
- Extra hash function → extra time

Hash Function, $h_i(x) = (h(x) + f(i)) \% \text{tableSize}$, where $f(i) = i \times h_2(x)$

Rehashing

- Increases the size of the hash table when load factor becomes higher than a cutoff or insertion fails.
- Must have been N/2 insertions since last rehash. Amortizing the O(N) cost over the N/2 prior insertions yields O(1) per insertion.

NP-Completeness

P: Set of decision problems that can be solved in polynomial time
NP: Set of decision problems with a polynomial time verification for a YES answer
EXP: Set of decision problems for which there exist exponential time algorithm
co-NP: Complements of decision problems in NP.

- P is a subset of NP
- P is a subset of EXP
- P is a subset of co-NP
- $x \in P \rightarrow \sim x \in P$

Problem A is polynomial time reducible to problem B, $A \leq_p B$
→ B is at least as hard as A
→ difficulty flows from A to B
→ Efficiency flows from B to A

If both $X \leq_P Y$ and $Y \leq_P X$, then $X \equiv_P Y$ (X can be solved in polynomial time iff Y can be)
NP-Hard: X is an NP-hard problem if for all $Y \in NP$, $Y \leq_P X$
NP-Complete: X is an NP-complete problem if $X \in NP$ and $Y \leq_P X$, for all $Y \in NP$.
NP-complete problems are:

- hardest in NP
- set of same difficulty level problems
- intersection of NP and NP-Hard

$P = NP \Rightarrow NP = co\text{-}NP$

NP-Completeness example problems

Conjunctive normal form (CNF): A propositional formula Φ that is a conjunction of clauses.
SAT: Given a CNF formula Φ, does it have a satisfying truth assignment?
3-SAT: SAT where each clause contains exactly 3 literals (and each literal corresponds to a different variable).

Independent Set

Given a graph G = (V, E) and an integer k, is there a subset of k (or more) vertices such that no two are adjacent?
Independent Set is NP-complete:
Independent Set is NP:
Given a proposed independent set S of vertices in a graph G and a positive integer k, we can verify in polynomial time whether S is an independent set with at least k vertices.
Independent Set is NP-Hard:
To prove that the Independent Set problem is NP-hard, we will reduce the 3-SAT problem to the Independent Set problem. This reduction will demonstrate that if we can solve Independent Set efficiently, we can solve 3-SAT efficiently as well.
3-SAT $\leq_P$ Independent Set
Construction: Given an instance Φ of 3-SAT, we construct an instance (G, k) of Independent Set. G contains 3 nodes for each clause, one for each literal. Connect 3 literals in a clause in a triangle. Connect literal to each of its negations.
Claim: Φ is satisfiable iff G contains an independent set of size $k = |\Phi|$
Proof:

1. Consider any satisfying assignment for Φ. Select one true literal from each clause/triangle. This is an independent set of size $k = |\Phi|$
2. Let S be an independent set of size k. S must contain exactly one node in each triangle. Set these literals to true (and remaining literals consistently). All clauses in Φ are satisfied.

This reduction shows that 3-SAT can be polynomially transformed into an Independent Set. This reduction implies that Independent Set is NP-hard.

Thus, the Independent Set problem is NP-Complete.

Vertex Cover

Given a graph G = (V, E) and an integer k, is there a subset of k (or fewer) vertices such that each edge is incident to at least one vertex in the subset?

Vertex Cover is NP-complete:

Vertex Cover is NP:

Given a set S in G, we can check in polytime if S is a vertex cover by going through all the edges if it is covered by S. Hence, the vertex cover is NP.

Vertex Cover is NP-Hard:

To prove that the Vertex Cover problem is NP-hard, we can reduce the Independent Set problem to the Vertex Cover problem. Actually, vertex cover and independent set reduce to one another. Independent Set $\equiv_P$ Vertex Cover.

Claim: We show S is an independent set of size k in G iff V − S is a vertex cover of size n – k.

Proof:

1. Let S be any independent set of size k in G. So, V − S is of size n – k.
   Consider an arbitrary edge (u, v) ∈ E. Since S is independent, either u ∉ S, or v ∉ S, or both. Which implies, either u ∈ V − S, or v ∈ V − S, or both. Thus, V − S covers (u, v).
2. Let V − S be any vertex cover in G of size n – k. S is of size k. Consider an arbitrary edge (u, v) ∈ E. V − S is a vertex cover, which means either u ∈ V − S, or v ∈ V − S, or both. This implies either u ∉ S, or v ∉ S, or both. Thus, S is an independent set.

This reduction shows that Independent Set can be polynomially transformed into Vertex Cover. Since Independent Set is NP-hard, this reduction implies that Vertex Cover is also NP-hard.

Thus, Vertex Cover is NP-complete.

Set Cover

Given a set U of n elements, a collection of S_1, S_2, …, S_m of subsets of U, is there a collection of at most k of these subsets whose union equals U?

Set cover is np-complete:

Set cover is NP:

The Set Cover problem is in NP because given a potential solution (a subset of sets) and a proposed certificate (the chosen sets), we can easily verify whether the certificate indeed covers all elements. This verification can be done in polynomial time.

Set cover is NP-hard:
To prove that the Set Cover problem is NP-complete, we will reduce the Vertex Cover problem to the Set Cover problem. Vertex Cover $\leq_P$ Set Cover.
Construction: Given a vertex cover instance G = (V, E) and k, we will construct a set cover instance (U, S, k). Here, universe U = E. Include one subset for each node $v \in V$: $S_v = \{e \in E : e \text{ is incident to } v\}$.
Claim: Given the vertex cover instance G = (V, E) and k, the set cover instance (U, S, k) has a set cover of size k iff G has a vertex cover of size k.
Proof:

1. Let $X \subseteq V$ be a vertex cover of size k in G. Then $Y = \{S_v : v \in X\}$ is a set cover of size k.
2. Let $Y \subseteq S$ be a set cover of size k in (U, S, k). Then $X = \{v : S_v \in Y\}$ is a vertex cover of size k in G.

This reduction shows that Vertex Cover can be polynomially transformed into Set Cover. Therefore, if we can solve Set Cover efficiently, we can solve vertex cover efficiently as well. Since Vertex Cover is NP-complete, this reduction implies that Set Cover is also NP-hard.
Thus, the Set Cover is NP-complete.

Hamiltonian Cycle

Given an undirected graph G = (V, E), does there exist a cycle Γ that visits every node exactly once?
Hamiltonian Cycle is NP-Complete:
Hamiltonian Cycle is NP:
Given a proposed Hamiltonian cycle C in a graph G, we can verify in polynomial time whether C is indeed a Hamiltonian cycle.
Hamiltonian Cycle is NP-Hard:
Directed Hamiltonian Cycle $\leq_P$ Hamiltonian Cycle
Construction: Given a directed graph G = (V, E), construct a graph G′ with 3n (in, out, connector) nodes.
Claim: G has a directed Hamilton cycle iff G′ has a Hamilton cycle.
Proof:

1. Suppose G has a directed Hamilton cycle Γ. Then G′ has an undirected Hamilton cycle (same order).
2. Suppose G′ has an undirected Hamilton cycle Γ′. Γ′ must visit nodes in G′ using one of the following two orders → (… out, connector, in, out, connector, in, …) or

(… in, connector, out, in, connector, out, …) Connector nodes in Γ′ comprise either a directed Hamilton cycle Γ in G, or reverse of one.

Traveling Salesperson Problem (TSP)

Given a set of cities and distance between every pair of cities, the problem is to find the shortest possible route that visits every city exactly once and returns to the starting point.

Traveling salesperson problem is NP-complete:

TSP is NP:

TSP can be verified in polynomial time (given a tour and weights, we can check if the tour is valid and its weight is at most |V|). So, TSP is in NP.

TSP is NP-hard:

We can achieve this by showing a reduction from the Hamiltonian Cycle Problem (HCP) to the Traveling Salesperson Problem (TSP). HCP $\leq_P$ TSP.

Construction: Given an instance of the Hamiltonian Cycle Problem, which is an undirected graph G, we will create an instance of the Traveling Salesperson Problem. Create a complete graph K based on the vertices of G. The vertices of G will become the cities in the TSP instance. Assign weights to the edges of K such that the weight of an edge between vertices u and v in K is 1 if there is an edge between u and v in G, and the weight is 2 otherwise.

Claim: G has a Hamiltonian cycle if and only if the TSP instance has a tour of total weight at most |V|.

Proof:

1. If G has a Hamiltonian cycle, then the corresponding tour in the TSP instance would visit each city exactly once, and since the edges in the tour correspond to edges in G, the total weight of the tour would be |V|.
2. If the TSP instance has a tour of total weight at most |V|, this means the tour visited every city exactly once. Because the weights are either 1 or 2, the tour must include exactly one edge between each pair of cities. If there is an edge of weight 1 between two cities, it corresponds to an edge in G. Thus, the tour corresponds to a Hamiltonian cycle in G.

Since the reduction can be performed in polynomial time and the properties hold, we've established that TSP is NP-hard.

Combining these two points, we conclude that the Traveling Salesperson Problem (TSP) is NP-complete. This completes the proof.

3-Coloring

Given an undirected graph G, can the nodes be colored with 3 different colors so that no adjacent nodes have the same color?

3-Coloring is NP-complete:

3-Coloring is NP:

This means that if someone gives us a proposed coloring of a graph, we can quickly verify whether it's a valid 3-coloring or not. This can be done in polynomial time by checking that no two adjacent vertices have the same color and that each vertex is colored with one of three colors.

3-Coloring is NP-Hard:

3-SAT $\leq_P$ 3-Coloring.

Construction:

We define nodes v_i and v_i' corresponding to each variable x_i and its negation x_i'. We also define three special nodes: True, False, Base. We connect each v_i's with the Base node. In any 3-coloring of G, the nodes v_i and v_i' must get different colors, and both must be different from Base. The nodes True, False, and Base must get all three colors in some permutation. This means that for each i, one of v_i and v_i' gets the True color, and the other gets the False color.

For each clause in the 3-SAT instance, we attach a six-node subgraph. Now suppose that in some 3-coloring of G all three of v_1, v_2' and v_3 are assigned the False color. Then the lowest two shaded nodes in the subgraph must receive the Base color, the three shaded nodes above them must receive, respectively, False, Base, and True. Hence there is no color that can be assigned to the topmost shaded node.

Claim:

The graph G is 3-colorable iff Φ is satisfiable.

Proof:

1. Suppose that there is a satisfying assignment for the 3-SAT instance. We define a coloring of G' by first coloring Base, True, and False arbitrarily with the three colors, then, for each i, assigning v_i the True color if $x_i = 1$ and the False color if $x_i = 0$. v_i' is assigned the only available color. It is thus possible to extend this 3-coloring into each six-node clause subgraph, resulting in a 3-coloring of all of G'.
2. Conversely, suppose G' has a 3-coloring. In this coloring, each node v_i is assigned either the True color or the False color; we set the variable x_i correspondingly. In each clause of the 3-SAT instance, at least one of the terms in the clause has the truth value T. For if not, then all three of the corresponding nodes have the False

color in the 3-coloring of G' and, as we have seen above, there is no 3-coloring of the corresponding clause subgraph consistent with this – a contradiction.

Since the reduction from 3-SAT to 3-Coloring can be done in polynomial time, this proves that 3-Coloring is NP-hard. Since we already know that it's in NP, we conclude that 3-Coloring is NP-complete.

Register Allocation

Assign program variables to machine registers so that no more than k registers are used and no two program variables that are needed at the same time are assigned to the same register.

Register Allocation is NP-Complete:

Register Allocation is NP-Hard:

3-Coloring $\leq_P$ k-Register-Allocation for any constant $k \geq 3$

Construction: Nodes are program variables; edge between u and v if there exists an operation where both u and v are live at the same time.

Claim: We can solve the register allocation problem iff the interference graph is k-colorable.

Proof:

1. If the interference graph can be colored with k colors, this implies that no two program variables (nodes) connected by an edge (i.e., interfering variables) are assigned the same color. Therefore, these variables can be assigned to different registers without any conflict. Thus, register allocation is possible using k or fewer registers in this case.
2. If the programs can be scheduled in k-registers, we can color corresponding nodes with k colors. Thus the interference graph would be k-colorable.

Since the ability to solve the register allocation problem is contingent on whether the interference graph can be k-colored, and 3-Coloring (which is NP-Complete) can be reduced to k-Register Allocation for any constant $k \geq 3$, it follows that Register Allocation is NP-hard.

And since it is in NP, register allocation is NP-complete.

Knapsack

Given a set of items X, weights $w_i \geq 0$, values $v_i \geq 0$, a weight limit W, and a target value V, is there a subset $S \subseteq X$ such that: $\sum w_i \leq W$ and $\sum v_i \geq V$, where, $i \in S$.

Knapsack in NP-Complete:

Knapsack in NP:

Given a set of items and a subset, we can check in polytime, if their total weight is less than W and value at least V.

Knapsack in NP-Hard:

Subset-Sum $\leq_p$ Knapsack

Construction: Given an instance of the Subset Sum Problem with a set of integers S = $\{a_1, a_2, \ldots, a_n\}$ and a target integer K, we can construct an instance of the Knapsack Problem. Create a Knapsack with a weight capacity of W = K. For each integer a_i in S, create an item with weight $w_i = v_i = a_i$.

Claim: There exists a subset of integers in S whose sum is exactly K in the Subset Sum instance, iff there exists a Knapsack solution in the constructed instance with a total value of at least K.

Proof:

1. If there exists a subset of integers in S whose sum is exactly K, you can select the corresponding items in the Knapsack instance to achieve a total value of K (since their weights and values are the same).
2. If there exists a Knapsack solution in the constructed instance with a total value of at least K, then there exists a subset of integers in S whose sum is exactly K in the Subset Sum instance.

Since Subset Sum is NP-complete, and we've shown a polynomial-time reduction from Subset Sum to the Knapsack decision problem, it follows that the Knapsack decision problem is NP-hard.

As it is also in NP, Knapsack is NP-Complete.

Coping With Hardness

Backtracking Algorithm

- Pruning the search space (in a tree like space)
- Incrementally grow a tree of partial solutions
- Rejecting a solution by looking at a small portion of it
- Check trees like DFS
- Applies to non-optimization problem

Example: 3-SAT, 4-SAT, n-queen problem, TSP, subset sum, graph coloring, hamiltonian cycle.

Branch and Bound Algorithm

- A generalization of the concept of Backtracking for optimization problems
- For each branch of the tree, we compute a rough bound on the solutions that can be obtained by exploring the solutions resulting from extending that branch
- This is the basis for eliminating some "non-promising" branches
- We do not explore the branch that won't lead us to better solutions that what we have already achieved
- Check trees in level order (BFS)
- It can solve minimization problems, not maximization problems

Example: 0-1 knapsack, TSP, job sequence.

Greedy

Greedy independent set algorithm on trees

The greedy algorithm finds a max-cardinality independent set in forests (and hence trees)

```
independentSetForest (F):
    S ← ∅
    while (F has at least 1 edge):
        Let v be a leaf node and let (u, v) be the lone edge incident to v
        S ← S ∪ {v}
        F ← F – {u, v}
    Return S ∪ {nodes remaining in F}
```

Time complexity: O(n)

Greedy vertex cover algorithm

Given a graph G = (V, E), find a min-size vertex cover

```
vertexCover (G):
    S ← ∅
    E′ ← E
    while E′ ≠ ∅:
        Let (u, v) ∈ E′ be an arbitrary edge
        S ← S ∪ {u} ∪ {v}
        Delete from E′ all edges incident to either u or v
    return S
```

Time complexity: O(m + n)

This greedy algorithm is not optimal.

Dynamic Programming

Weighted independent set on trees

Given a tree and node weights $w_v \geq 0$, find an independent set S that maximizes $\Sigma_{v \in S} w_v$

weightedIndependentSetTree (T):

Root the tree T at any node r

$S \leftarrow \varnothing$

For each node u of T in postorder/topological order:

if (u is a leaf node)

$M_{in}[u] = w_u$

$M_{out}[u] = 0$

else

$M_{in}[u] = w_u + \Sigma_{v \in children(u)} M_{out}[v]$

$M_{out}[u] = \Sigma_{v \in children(u)} \max\{M_{in}[v], M_{out}[v]\}$

return $\max\{M_{in}[r], M_{out}[r]\}$

Time complexity: O(n)

Knapsack

DP-I

OPT(i, w) = max value subset of items 1,2, …, i with remaining weight limit w

OPT(i, w) →

0	when i = 0
OPT(i - 1, w)	when $w_i > w$
$\max\{OPT(i - 1, w), OPT(i - 1, w - w_i) + v_i\}$	otherwise

Time complexity: O(nW)

DP-II

OPT(i, v) = min weight of a knapsack for which we can obtain a solution of value ≥ v using a subset of items 1, 2, …, i

Optimal value is the largest value v such that $OPT(n, v) \leq W$

OPT(i, v) →

0	when $v \leq 0$
∞	when i = 0 and v > 0
$\min\{OPT(i - 1, v), OPT(i - 1, v - v_i) + w_i\}$	otherwise

Time complexity: $O(n^2 v_{max})$

Others

Chaitin's Algorithm (k-coloring)

Chaitin's algorithm produces a k-coloring of any graph with max degree k – 1

```
vertexColor (G, k):
    while (G is not empty):
        Pick a node v with fewer than k neighbors
        Push v on stack
        Delete v and all its incident edges
    while (stack is not empty):
        Pop next node v from the stack
        Assign v a color different from its already colored neighboring nodes
```

Approximation Algorithm

- Runs in polynomial time.
- Has a performance guarantee.

Approximation algorithm vs Heuristic Algorithm:

Heuristics are problem-solving methods that use practical, rule-of-thumb techniques to find good, but not necessarily optimal, solutions quickly. Heuristics do not provide any formal guarantee about the quality of the solution. They aim to find a reasonably good solution within a reasonable amount of time.

Approximation algorithms are designed to find a solution that is guaranteed to be close to the optimal solution. They aim to provide a provable upper bound on how far their solution is from the optimal. Approximation algorithms provide formal bounds on the quality of the solution they produce. For example, a 2-approximation algorithm guarantees that the solution it finds is at most twice the optimal. Approximation algorithms are generally more efficient than exact algorithms but can be slower than pure heuristics because they aim for a balance between quality and speed.

Minimum Vertex Cover

The greedy algorithm is a 2-approximation algorithm.

Knapsack

Rounding algorithm:

- Round all values up to lie in a smaller range.
- Run dynamic programming algorithm II on rounded/scaled instances.

- Return optimal items in a rounded instance.

For any $\varepsilon > 0$, the rounding algorithm computes a feasible solution whose value is within a $(1 + \varepsilon)$ factor of the optimum in $O(n^3/\varepsilon)$

Metric TSP

Algorithm: 2-Approximation for Metric TSP

1. Compute the Minimum Spanning Tree (MST) of the given graph.
2. Duplicate every edge in the MST, forming a Eulerian multigraph.
3. Find an Eulerian tour.
4. Convert the Eulerian tour into a Hamiltonian tour by skipping repeated vertices.
5. The resulting tour is a 2-approximation of the Metric TSP.

Explanation:

The MST of a graph is a tree that connects all vertices with the minimum possible total edge weight. Duplicating the edges in the MST allows us to visit each city once and return to the starting city, forming a Eulerian tour. This ensures that we visit every edge of the original graph. The Eulerian tour is converted into a Hamiltonian tour by skipping any repeated vertices, as we don't need to visit a city more than once. The total length of the Hamiltonian tour is at most twice the weight of the MST (by the triangle inequality), making it a 2-approximation.

This algorithm guarantees a tour with a length at most twice the optimal tour length.

Exponential Algorithm

TSP

TSP can be solved in $O(n^2\, 2^n)$ time (DP)

$c(s, v, X)$ = cost of cheapest path between s and $v \neq s$ that visits every node in X exactly once (and uses only nodes in X).

There are $\leq n\, 2^n$ subproblems and they satisfy the recurrence:

$c(s, v, X) \rightarrow$

$w(s, v)$	when $\lvert X\rvert = 2$
$\min_{u \in X\setminus\{s,v\}} c(s, u, X\setminus\{v\}) + w(u, v)$	otherwise